Conceptual Architecture: WezTerm

CISC 326/322 - Software Architecture

Assignment 1

URL:

Youtube link : https://youtu.be/TgZfyRJAR5o?si=QJmEEkHes204yvr9

Website : https://aakwsp.github.io/goober-cats/

meow meow meow meow meow meow meow meow meow meow meow meow meow
meow meow meow meow meow meow meow meow meow meow meow meow meow
meow meow meow meow meow meow meow meow meow meow meow meow meow
meow meow meow meow meow meow meow meow meow meow meow meow meow
meow meow meow meow meow meow meow meow meow meow meow meow meow

Group Members:



Julia Lukaszun - Presenter

Nathan Beard - Presenter

Moshi Moshiri - Group Leader

Amro Shanaah

Mark Talinio

Matthew Wong

Tim Rostorhuiev

Overview

- WezTerm is an open-source, cross-platform terminal emulator and multiplexer implemented in the Rust programming language. compatible with multiple operating systems including Linux, macOS, Windows, FreeBSD and NetBSD. In addition to providing a traditional terminal, WezTerm allows users to organize terminal sessions into panes, tabs, windows and workspaces. It can also connect the user interface to different execution environments through its multiplexer architecture.
- Compatible with multiple operating systems including Linux, macOS, Windows, FreeBSD and NetBSD.

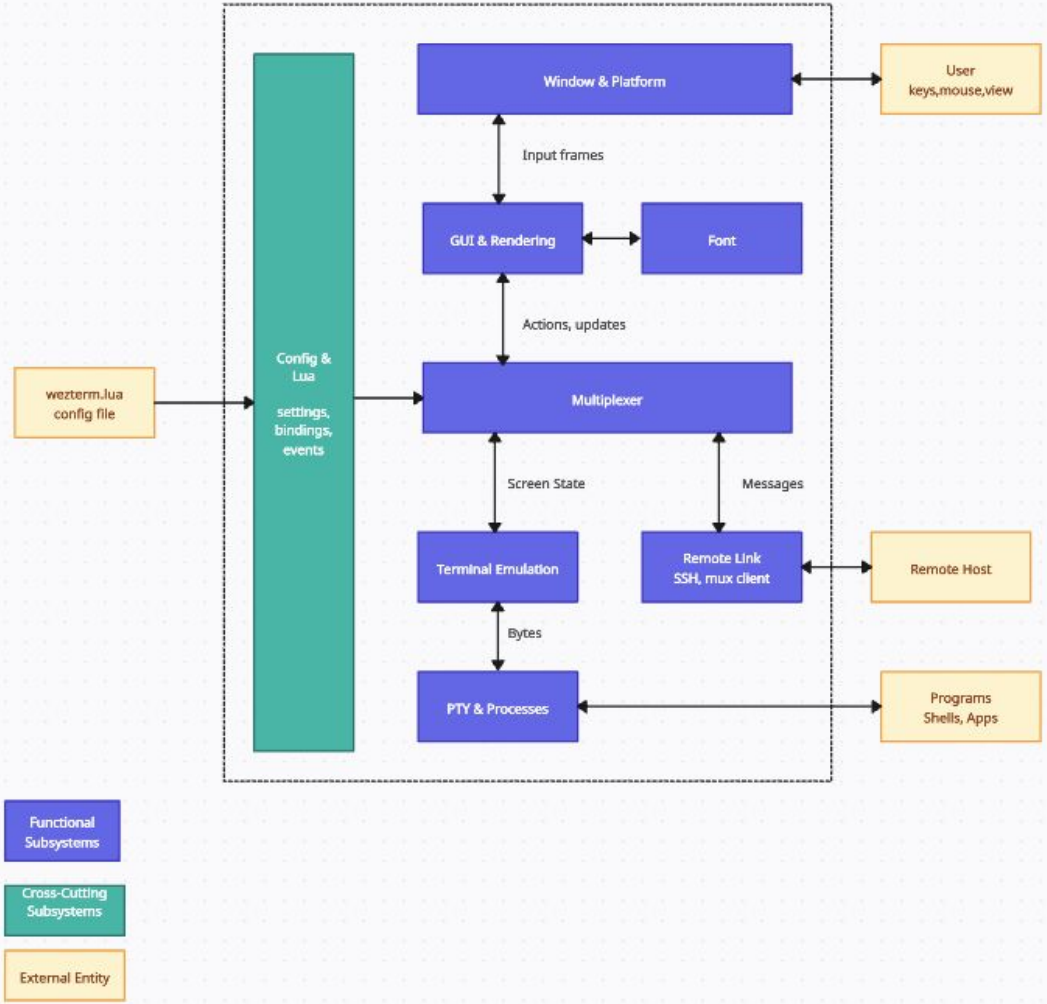
How We Derived the Architecture

- Sources include:
  - WezTerm documentation,
  - Changelog
  - Config page
  - GitHub releases
- Our steps:
  - (1) Read the documentation
  - (2) Identify responsibilities
  - (3) Group them into components
  - (4) Trace flows
  - (5) Draw diagrams

WezTerm's Architectural Style

- ➔ WezTerm possesses a Layered architectural style
  - ◆ To separate responsibilities between UI, terminal and session management, rendering and platform-specific services.
- ➔ Benefits:
  - ◆ Lower coupling
  - ◆ Swappable parts
  - ◆ Easier testability
- ➔ Trade-offs:
  - ◆ Layers are not fully independent → integration testing needed
- ➔ Alternative: Monolithic Architectural Style
  - ◆ Able to simplify communication between responsibilities
  - ◆ Increases coupling
  - ◆ Harder to update individual components

Functionality of the Top Level Subsystems

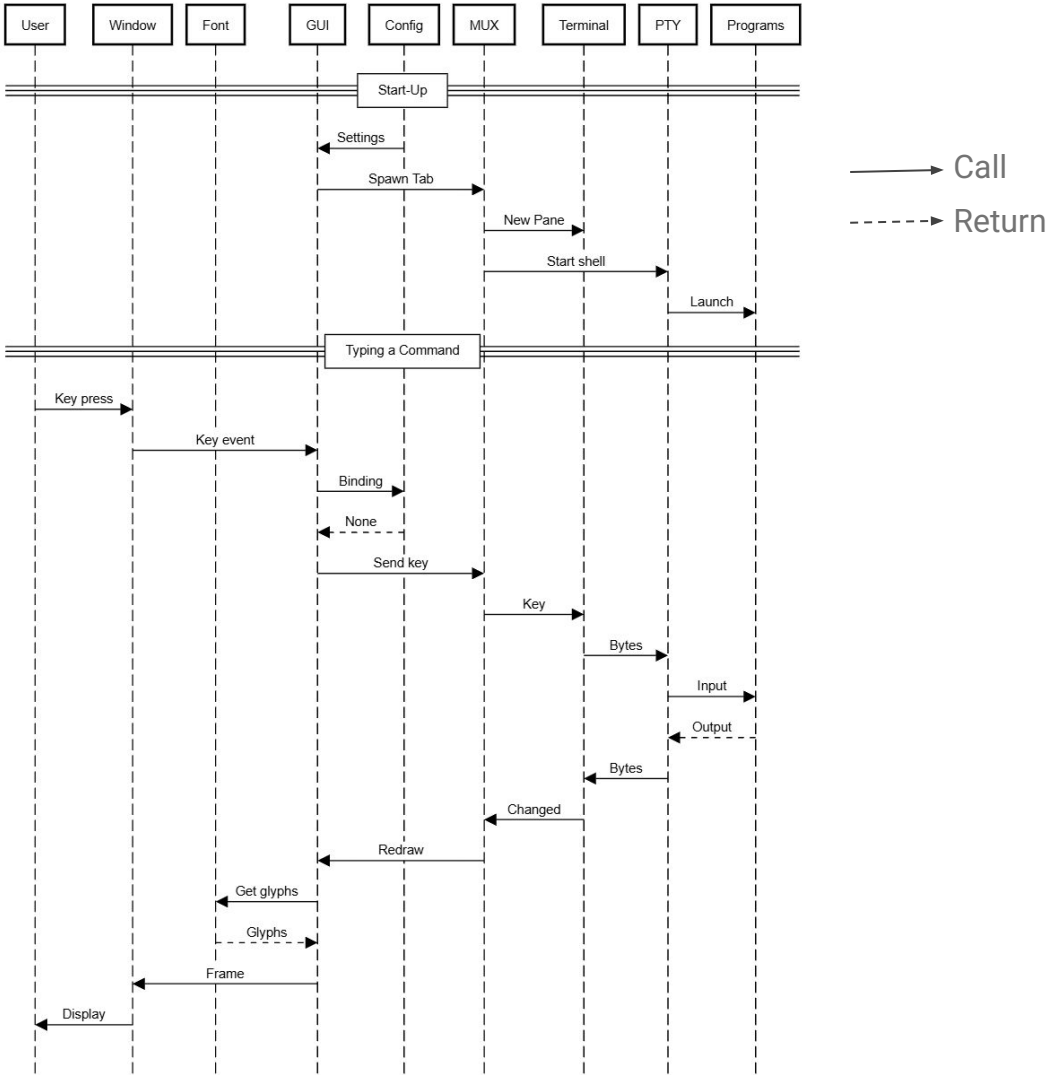


Start-up

1. Config passes the loaded settings to the GUI.
2. The GUI asks the MUX to create the first tab.
3. The MUX creates a Pane in the Terminal emulation and has the PTY start the user's shell.

Typing a command

1. A Keypress goes from Window to the GUI.
2. The GUI checks Config for a key binding. In this instance there isn't one.
3. The Key travels down through the MUX, Terminal and PTY to the program.
4. The Programs output comes back up the same layers. THe MUX tells the GUI to Redraw
5. The GUI gets glyph from Font, builds a frame, and Window displays it to the user.



Rationale for the Subsystem Interactions

The primary goal for the interaction between WezTerm's subsystems is for each subsystem should be able to handle one kind of information and know as little as possible about the others.

Window to GUI: Keystrokes are received by the the window subsystem and passed to the GUI as a platform independent key events.

GUI to Config: The GUI checks ever key against the configuration before it passes on. It performs this because WezTerm has its own actions.

GUI to MUX: Unbound keys are sent to the multiplexer, not directly to the terminal. It communicates the changes to the MUX. It does this because the GUI does not know where the pane is. The MUX decides if the pane belongs to a local program or a remote server.

MUX to Terminal to PTY:

The terminal emulation subsystem converts keys into byte sequences that programs expect, and the PTY subsystem delivers them.

These are kept separate because they are changed for different reasons.

Terminal emulation follows terminal standards that are common on every platform. The PTY depends on the operating system, using pseudo-terminals on Unix and ConPTY on Windows.

Keeping them apart allows the terminal emulation to be tested without starting a real program.

Rationale for the Subsystem Interactions

Programs back to Terminal:

Program output is interpreted only inside the terminal emulation subsystem, which keeps the authoritative model of the screen.

The other subsystems read that model instead of the raw output. This allows the escape sequence handling to exist only in one place.

GUI to Font:
Font loading and text shaping are separate from rendering. Keeping them within their own subsystem allows the results to be cached then reused, so the GUI only asks for the glyphs it has not drawn yet.

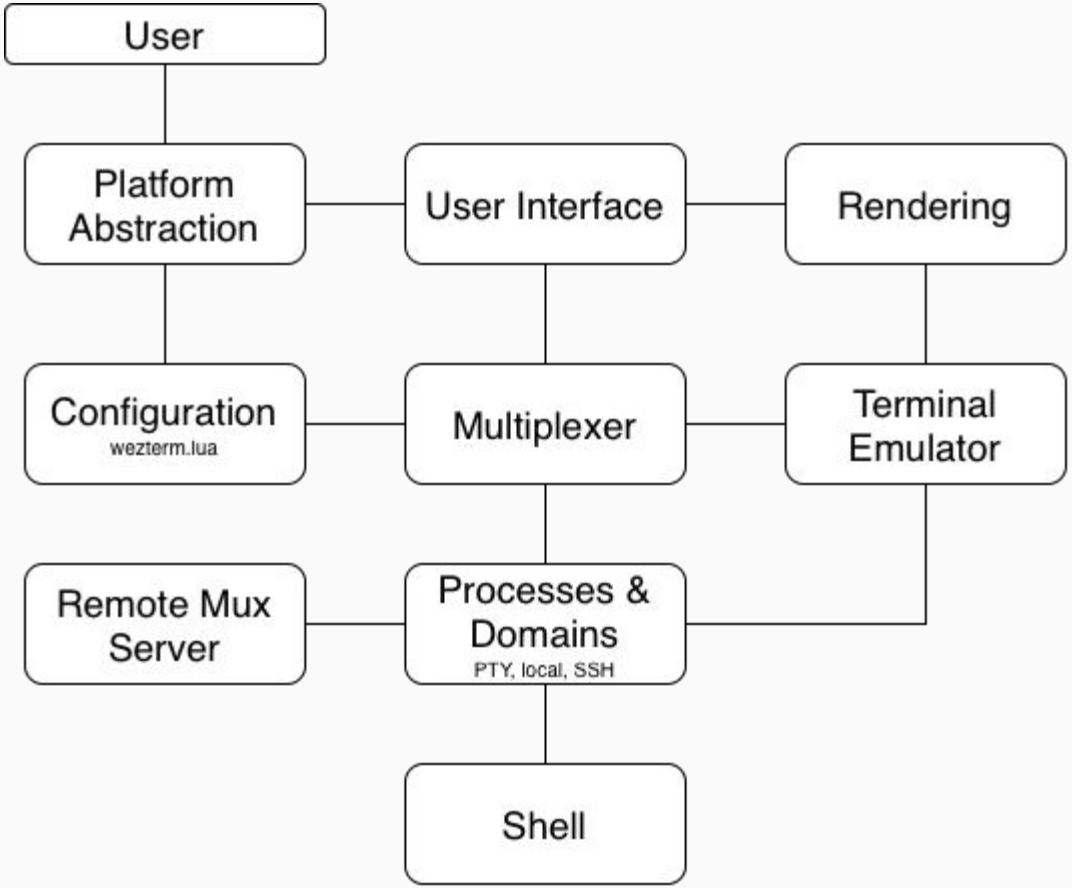
Terminal to MUX to GUI:

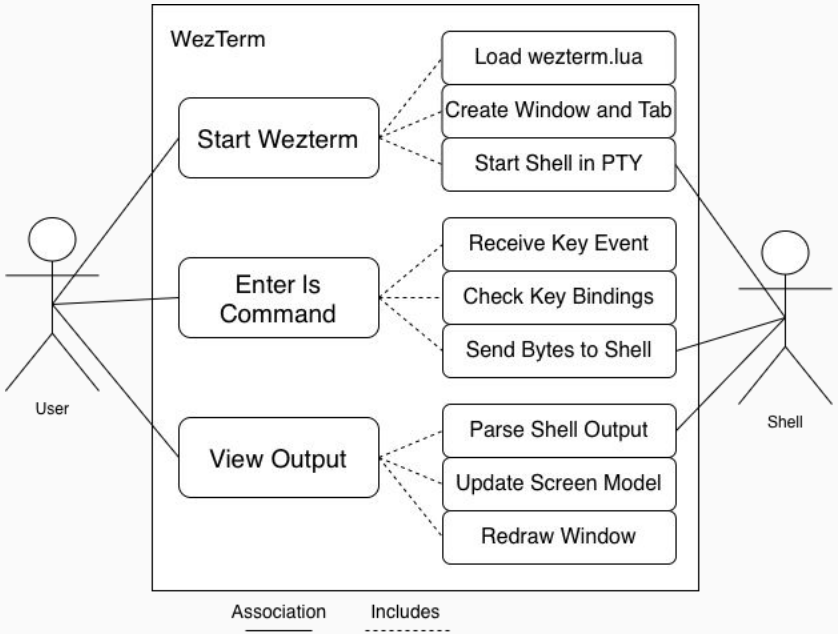
The terminal communicates to the MUX that the pane has changed and the MUX notifies the GUI to redraw. This approach keeps the lower layers independent of the display.

The same terminal code can run inside the multiplex server, and allows the GUI to combine many changes into a single redraw.

GUI to Window:
Only the GUI draws, and only through the window subsystem's GPU. This means rendering tech, such as Open GL or WebGPU, can change without affecting another subsystem.

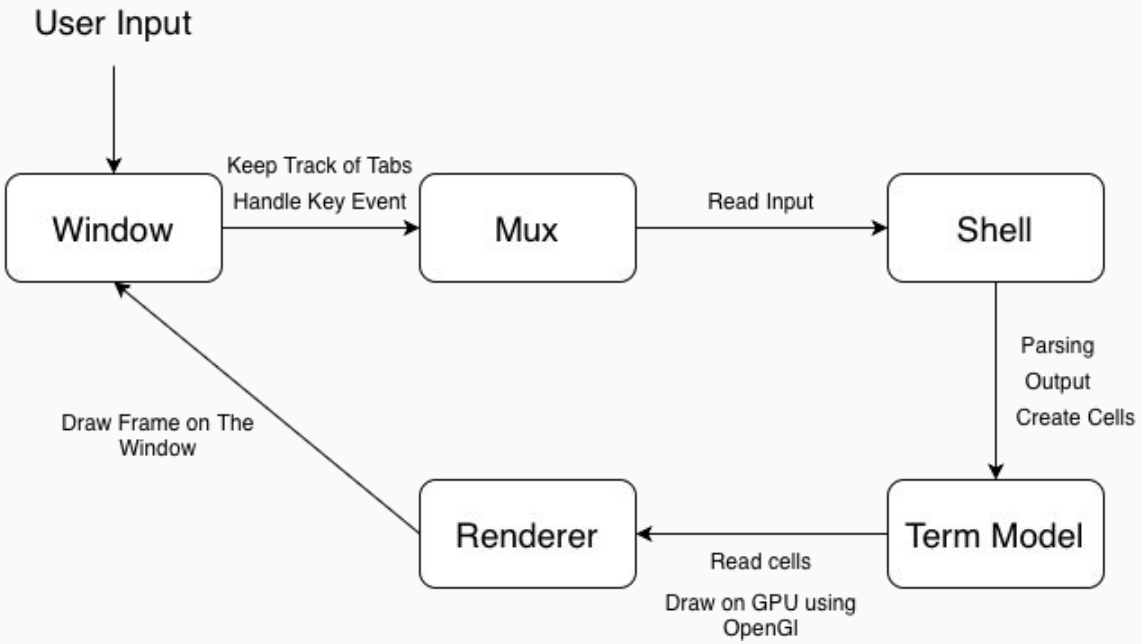
The 10 Components





Use Case: User running a command

Dependency Diagram



Concurrency and Performance

- Concurrency:
  - Multiple panes, tabs, windows
  - Each pane runs independently
  - Busy pane does not freeze UI
- Performance-Critical Components:
  - Rendering:
    - Must redraw everything
    - Makes the program smooth
  - Terminal Emulator:
    - Keep up with large amount of output
    - Remote domains depend on speed of the network
  - Collaborated about 9%

AI Teammate

- Model selected: Claude Opus 5.5
  - More reliable
- Tasks:
  - Architectural analysis
  - Flow reasoning
  - Concept clarification
- Prompting Strategy:
  - Questions → Prompts → Answers → Verification
- Quality Control:
  - Check claims against source material
- Impact:
  - Collaborated about 9%



Lesson Learned: not simple!

- ➔ Creating a conceptual architecture from documentation is difficult
- ➔ Importance of separating ‘similar’ concepts
  - ◆ Ex. terminal emulation and rendering
- ➔ What we would do next:
  - ◆ Figure out major architecture concepts early
  - ◆ Dive into detailed sections later

Conclusion

- WezTerm uses a Layered architecture
- Multiplexer domains give it client-server style
- Evolution stays within single components
- Main limitation:
  - The Development Process
- Next step:
  - Compare the source code to the conceptual architecture